

Exam rules

The exam consists of two separate parts.

- Part I (about 40 minutes): two theoretical questions (12 points) about subjects covered during lectures, such as time evaluation and hardware cost in modern architectures. You are not permitted to use any materials and software, including the calculator.
- Part II (about 90 minutes): Development of a program in ARM assembly and C language running on the LPC1768 board (18 points). At the beginning of this part, you can copy the following materials from your personal USB drive:
 - slides
 - user manual of the board
 - project templates: You may use either the official templates provided by the instructors or a personal template that you have created yourself. As the name suggests, a personal template may be used only by its creator and must not be shared with other students.
 - solutions of lab projects.

After the materials are copied to the Labinf PC, the USB drives must be disconnected and must not be used during the exam.

The duration of each part may slightly change depending on the difficulty of the exercises.

Furthermore, two projects are assigned in the last weeks of the course: they are reserved to students who uploaded the solutions of regular laboratories on time. For each project solution submitted on time, the student can get up to 2 extra points (up to 4 extra points for both projects).

In the very few days after the exam, the students have to self-correct their programs, starting from the solution completed during part II. Each student has to upload to the teaching portal (in the “Elaborati” section):

- The .zip of the new version of the Keil project
- The list of ALL changes, in the form of a .doc file with the following notation:
 - the written-exam-version of the developed functions is in black
 - new code is in green background color
 - the deletions are in background red color
 - instruction changes are in yellow background color

For example: unchanged text, new, deleted text, moved or modified text. See the following section for more detailed examples.

Not complying with this format and/or not marking ALL errors/ changes will automatically void the exam. Note: you have to report changes only for the developed functions (please avoid including the whole template in the .doc file).

The deadline for uploading the files will be timely communicated after the exam. If the files are not uploaded in time, the student will be rejected. If the self-corrected solution produces errors during the compiling phase or is incomplete, the student will be rejected.

The exam is passed if the student obtains:

- At least 6 points for part I
- At least 9 points for part II
- At least 18 points overall (part I + part II).

Then, the extra points will be added to the final score. Laude will be granted if the mark is higher than or equal to 33 (>32.5). The teachers have the right to proceed with the oral exam if they deem further study appropriate.

How to highlight changes in code

Very small edit

`for(int i=0; i<10; i++)` 0 was removed
`for(int i=0; i<100; i++)` 0 was added

Small edit

`for(float i=0; ...)` previously float, now int
`for(int i=0; ...)`

Substantial edit

`for(float i=0; i<10; i++)`
`for(int i=0; i<100; i++)` too many changes, better to rewrite

Instruction swap

`Instruction A` A and B were swapped.

`Instruction B`

other instructions

`Instruction A`

`Instruction B`

In case the inversion is not immediate to see, clarify it with a comment.

Instruction replacement in a block

Instr. 1 2 and 3 are removed, 4-5-6 are added
`Instr. 2`
`Instr. 3`
`Instr. 4`
`Instr. 5`
`Instr. 6`
Instr. 7

Simple deletion or addition are subcases of this.

Comments

No need to highlight them, but keep them short and clear if you think they will help us understand why you removed/added something.

Information Provided During the Exam

Note: This is a reproduction of the exam's first page. Since it contains important information, it is included here for your reference.

Please read the following notes before proceeding

- 1) It is recommended to save the project on the C drive during development. Once you have completed your work, you must save the project **inside the exam folder** on the Z drive (the same folder where you found this document). Projects saved elsewhere will not be collected and will be permanently lost.
- 2) The assembly subroutine developed in the first exercise must comply with the ARM Architecture Procedure Call Standard (AAPCS) standard (in terms of parameter passing, returned value, callee-saved registers).
- 3) If you prefer, you can begin with the second exercise. Since it requires calling the assembly subroutine from the first exercise, you can create a stub in the assembly file to proceed. For example:

```
subroutineName    PROC
    EXPORT subroutineName
    MOV r0, #10      ; example of a return value
    BX LR
    ENDP
```

- 4) To earn points on both exercises, it is recommended that you write at least some code for each.
- 5) The final project must run on the actual board.

Special characters with Italian keyboards

- Square brackets []
Left bracket: ATL GR + [
Right bracket: ALT GR +]
- Curly brackets { }
Left bracket: LSHIFT + ALT GR + [
Right bracket: LSHIFT + ALT GR +]
- Hash # : ALT GR + à

Bitwise operations in C

```
Left shift by N bits    =    A<<N
Right shift by N bits   =    A>>N
A XOR B                 =    A^B
A AND B                 =    A&B    (&& is the logical AND)
A OR B =                A|B    (|| is the logical OR)
```